MongoDB Administration

Complete Operational Guide

Installation • Configuration • CRUD • Backup & Recovery
Performance Tuning • Troubleshooting • Security • Replication

Target Platform: AlmaLinux 8 / 9 — Private Cloud VMs — /apps Filesystem

Table of Contents

1 Installation and Configuration on AlmaLinux 8/9

1.1 Prerequisites and Planning

Before installing MongoDB, ensure the target VM meets minimum requirements. MongoDB Community Edition 7.x (current stable) requires a 64-bit x86_64 processor, at least 4 GB RAM (8 GB+ recommended for production), and XFS filesystem for the data directory (ext4 is supported but XFS is strongly preferred for WiredTiger).

Filesystem Layout (Private Cloud /apps)

All MongoDB data, logs, and configuration will reside under the /apps filesystem per organizational standards.

Mount Point / Directory	Purpose	Recommended Size
/apps/mongodb/data	Database data files (dbPath)	100 GB+ (depends on data volume)
/apps/mongodb/log	Log files	10–20 GB
/apps/mongodb/config	Custom configuration files	100 MB
/apps/mongodb/backups	Local backup staging area	2x data size
/apps/mongodb/keyfile	Replica set authentication keyfile	10 MB

1.2 MongoDB RPM Packages — What Gets Installed

MongoDB is distributed as RPM packages for RHEL-based systems (AlmaLinux, Rocky, CentOS). Understanding the individual RPM components is essential for installation, troubleshooting, and upgrades.

RPM Package Name	Provides	Key Files Installed
mongodb-org	Meta-package (pulls in all below)	None — dependency wrapper only
mongodb-org-server	mongod daemon (the database server)	/usr/bin/mongod, /etc/mongod.conf, /usr/lib/systemd/system/mongod.service
mongodb-org-mongos	mongos query router (sharding)	/usr/bin/mongos
mongodb-mongosh	Modern interactive shell (mongosh)	/usr/bin/mongosh
mongodb-org-tools	Meta-package for CLI tools (pulls in below)	None — dependency wrapper
mongodb-database-tools	mongodump, mongorestore, mongoexport, mongoimport, mongostat, mongotop, bsondump	/usr/bin/mongodump, /usr/bin/mongorestore, etc.

Inspecting Installed RPM Packages

```
# List all installed MongoDB RPMs
rpm -qa | grep -i mongo
```

```
# Detailed info on a specific package
rpm -qi mongodb-org-server

# List all files owned by a package
rpm -ql mongodb-org-server

# Which package owns a specific file?
rpm -qf /usr/bin/mongod

# Verify package integrity (check for modified files)
rpm -V mongodb-org-server
```

1.3 Installation Method A — YUM/DNF Repository (Recommended)

This is the standard and recommended method. MongoDB provides an official YUM repository for RHEL/AlmaLinux. The repo-based approach handles dependency resolution and GPG signature verification automatically.

Step 1: Create the Repository File

```
# Create the MongoDB 7.0 repo file
sudo tee /etc/yum.repos.d/mongodb-org-7.0.repo <<'EOF'
[mongodb-org-7.0]
name=MongoDB Repository
baseurl=https://repo.mongodb.org/yum/redhat/$releasever/mongodb-org/7.0/x86_64/
gpgcheck=1
enabled=1
gpgkey=https://pgp.mongodb.com/server-7.0.asc
EOF

# Verify the repository is visible
sudo dnf repolist | grep mongodb

# List available MongoDB packages from the repo
sudo dnf list available mongodb-org*
```

NOTE

For AlmaLinux 8 use $releasever=8; for AlmaLinux 9 use $releasever=9. The variable auto-resolves.

Step 2: Install via DNF/YUM

```
# Install ALL MongoDB components (recommended)
sudo dnf install -y mongodb-org

# This meta-package installs:
#   mongodb-org-server       — mongod daemon
#   mongodb-org-mongos       — mongos router (for sharding)
#   mongodb-org-tools        — mongodump, mongorestore, etc.
#   mongodb-mongosh          — modern mongo shell (mongosh)
#   mongodb-database-tools   — CLI tools (export, import, stat, top)
```

```
# Verify installed version
mongod --version
mongosh --version

# Verify all RPMs installed correctly
rpm -qa | grep mongo | sort
```

Step 3: Install a Specific Version (Version Pinning)

```
# Install a specific MongoDB release (e.g., 7.0.12)
sudo dnf install -y \
    mongodb-org-7.0.12 \
    mongodb-org-database-7.0.12 \
    mongodb-org-server-7.0.12 \
    mongodb-org-mongos-7.0.12 \
    mongodb-org-tools-7.0.12

# Pin MongoDB packages to prevent accidental upgrades via dnf update
# Add to /etc/yum.conf or /etc/dnf/dnf.conf:
echo "exclude=mongodb-org,mongodb-org-server,mongodb-org-mongos,mongodb-org-tools,mongodb-org-database,mong
    | sudo tee -a /etc/yum.conf

# Alternatively, use dnf versionlock plugin
sudo dnf install -y python3-dnf-plugin-versionlock
sudo dnf versionlock add mongodb-org mongodb-org-server mongodb-org-mongos
sudo dnf versionlock list
```

1.4 Installation Method B — Manual RPM Download and Install

In air-gapped or restricted environments where the VMs cannot reach repo.mongodb.org, download the RPM files on an internet-connected machine, transfer them, and install locally with rpm or dnf localinstall.

Step 1: Download RPMs (from an internet-connected machine)

```
# MongoDB RPM download URL structure:
#   https://repo.mongodb.org/yum/redhat/<RHEL_VER>/mongodb-org/<MONGO_VER>/x86_64/RPMS/
#
# Example for MongoDB 7.0 on AlmaLinux 9 (RHEL 9):
MONGO_VER="7.0.12"
RHEL_VER="9"
BASE_URL="https://repo.mongodb.org/yum/redhat/${RHEL_VER}/mongodb-org/7.0/x86_64/RPMS"

# Download individual RPMs
mkdir -p /tmp/mongodb-rpms && cd /tmp/mongodb-rpms

curl -O ${BASE_URL}/mongodb-org-server-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE_URL}/mongodb-org-mongos-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE_URL}/mongodb-org-tools-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE_URL}/mongodb-org-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
```

```
# mongosh and database-tools may have different version numbers
# Check the RPMS directory listing for exact filenames
curl -O ${BASE_URL}/mongodb-mongosh-<ver>.x86_64.rpm
curl -O ${BASE_URL}/mongodb-database-tools-<ver>.x86_64.rpm

# Verify downloaded RPMs (GPG signature check)
rpm --import https://pgp.mongodb.com/server-7.0.asc
rpm -K *.rpm
```

Step 2: Transfer to Target Server

```
# SCP the RPMs to the target private-cloud VM
scp /tmp/mongodb-rpms/*.rpm admin@target-vm:/tmp/mongodb-rpms/

# Or use a shared NFS/CIFS mount, USB, etc.
```

Step 3: Install RPMs Locally

```
# Method 1: Using dnf localinstall (resolves local dependencies)
cd /tmp/mongodb-rpms
sudo dnf localinstall -y *.rpm

# Method 2: Using rpm directly (no automatic dependency resolution)
sudo rpm -ivh mongodb-org-server-*.rpm \
              mongodb-org-mongos-*.rpm \
              mongodb-org-tools-*.rpm \
              mongodb-database-tools-*.rpm \
              mongodb-mongosh-*.rpm \
              mongodb-org-7*.rpm

# Verify installation
rpm -qa | grep mongo | sort
mongod --version
mongosh --version
```

TIP

Always prefer 'dnf localinstall' over raw 'rpm -ivh' as it automatically resolves and installs any missing OS-level dependencies (like openssl-libs, cyrus-sasl, etc.).

Step 4: Create a Local RPM Repository (Optional — for multiple servers)

```
# On a central staging server, set up a local repo
sudo dnf install -y createrepo

# Place all MongoDB RPMs in a directory
sudo mkdir -p /apps/repos/mongodb
sudo cp /tmp/mongodb-rpms/*.rpm /apps/repos/mongodb/
```

```
# Generate repo metadata
sudo createrepo /apps/repos/mongodb/

# On each target VM, create a repo file pointing to the local repo
sudo tee /etc/yum.repos.d/mongodb-local.repo <<'EOF'
[mongodb-local]
name=MongoDB Local Repository
baseurl=file:///apps/repos/mongodb/
# Or use http if served via httpd/nginx:
# baseurl=http://repo-server.internal/mongodb/
gpgcheck=0
enabled=1
EOF

# Now install normally
sudo dnf install -y mongodb-org
```

1.5 Prepare the /apps Filesystem Directories

```
# Create directory structure
sudo mkdir -p /apps/mongodb/{data,log,config,backups,keyfile}

# Set ownership to mongod user (created by the RPM)
sudo chown -R mongod:mongod /apps/mongodb

# Set permissions
sudo chmod 750 /apps/mongodb/data
sudo chmod 750 /apps/mongodb/log
sudo chmod 700 /apps/mongodb/keyfile
```

1.6 Configure MongoDB — mongod.conf

Replace the default /etc/mongod.conf with a production-ready configuration that points to /apps.

```
# /etc/mongod.conf  (YAML format)
# ■■ Storage ■■
storage:
  dbPath: /apps/mongodb/data
  journal:
    enabled: true
  wiredTiger:
    engineConfig:
      cacheSizeGB: 2            # Set to ~50% of RAM minus 1 GB
      journalCompressor: snappy
    collectionConfig:
      blockCompressor: snappy

# ■■ Logging ■■
systemLog:
  destination: file
  path: /apps/mongodb/log/mongod.log
```

```
  logAppend: true
  logRotate: reopen            # Allows external logrotate

# ■■ Network ■■
net:
  port: 27017
  bindIp: 0.0.0.0             # Bind to all interfaces (restrict via firewall)
  maxIncomingConnections: 500

# ■■ Process Management ■■
processManagement:
  timeZoneInfo: /usr/share/zoneinfo
  fork: false                  # Let systemd manage the process

# ■■ Security (enable after initial setup) ■■
#security:
#  authorization: enabled
#  keyFile: /apps/mongodb/keyfile/mongo-keyfile

# ■■ Operation Profiling ■■
operationProfiling:
  mode: slowOp
  slowOpThresholdMs: 100

# ■■ Replication (enable for replica sets) ■■
#replication:
#  replSetName: rs0
#  oplogSizeMB: 2048
```

1.7 SELinux Configuration

On AlmaLinux with SELinux enforcing, MongoDB needs custom policy adjustments for the /apps path.

```
# Check current SELinux status
getenforce

# Allow mongod to use /apps/mongodb directories
sudo semanage fcontext -a -t mongod_var_lib_t "/apps/mongodb/data(/.*)?"
sudo semanage fcontext -a -t mongod_log_t "/apps/mongodb/log(/.*)?"
sudo restorecon -Rv /apps/mongodb/

# If semanage is not available, install it:
sudo dnf install -y policycoreutils-python-utils

# Allow mongod to use non-default ports (if changed from 27017)
sudo semanage port -a -t mongod_port_t -p tcp 27017
```

1.8 Firewall Configuration

```
# Open MongoDB port
sudo firewall-cmd --permanent --add-port=27017/tcp
```

```
sudo firewall-cmd --reload

# Verify
sudo firewall-cmd --list-ports
```

1.9 Enable and Start the Service

```
# Enable at boot and start
sudo systemctl enable mongod
sudo systemctl start mongod

# Check status
sudo systemctl status mongod

# Verify connectivity
mongosh --eval "db.runCommand({ping:1})"
```

1.10 Systemd Drop-in Override (Recommended)

Create a systemd override to ensure correct LimitNOFILE and LimitNPROC values for production workloads.

```
sudo mkdir -p /etc/systemd/system/mongod.service.d

sudo tee /etc/systemd/system/mongod.service.d/override.conf <<'EOF'
[Service]
LimitNOFILE=64000
LimitNPROC=64000
# Auto-restart on failure
Restart=on-failure
RestartSec=5
EOF

sudo systemctl daemon-reload
sudo systemctl restart mongod
```

1.11 Kernel and OS Tuning

```
# Disable Transparent Huge Pages (THP) — required by MongoDB
sudo tee /etc/systemd/system/disable-thp.service <<'EOF'
[Unit]
Description=Disable Transparent Huge Pages
After=sysinit.target local-fs.target

[Service]
Type=oneshot
ExecStart=/bin/sh -c "echo never > /sys/kernel/mm/transparent_hugepage/enabled"
ExecStart=/bin/sh -c "echo never > /sys/kernel/mm/transparent_hugepage/defrag"
```

```
[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now disable-thp.service

# Increase vm.swappiness (MongoDB recommends 1)
echo "vm.swappiness = 1" | sudo tee -a /etc/sysctl.d/99-mongodb.conf

# Increase file descriptor limits in /etc/security/limits.d/
sudo tee /etc/security/limits.d/mongod.conf <<'EOF'
mongod soft nofile 64000
mongod hard nofile 64000
mongod soft nproc  64000
mongod hard nproc  64000
EOF

sudo sysctl -p /etc/sysctl.d/99-mongodb.conf
```

WARNING

Transparent Huge Pages MUST be disabled. MongoDB's memory allocator (WiredTiger) conflicts with THP and can cause severe latency spikes and excessive memory usage.

2 Database Objects — Collections, Indexes, Views, Schemas

2.1 MongoDB Terminology Mapping (RDBMS → MongoDB)

RDBMS Concept	MongoDB Equivalent	Description
Database	Database	Logical grouping of collections
Table	Collection	Group of BSON documents (like rows)
Row	Document	A single BSON/JSON record
Column	Field	Key-value pair within a document
Primary Key	_id field	Auto-generated ObjectId or user-defined
Index	Index	B-tree indexes on fields
View	View	Read-only, computed from aggregation pipeline
JOIN	$lookup	Aggregation stage to join collections
Schema	Validation Rules	Optional JSON Schema enforcement

2.2 Creating and Managing Databases

```
# Connect to MongoDB
mongosh

# Switch to / create a database (created on first write)
use appdb

# Show current database
db

# List all databases (only shows those with data)
show dbs

# Drop a database
use appdb
db.dropDatabase()
```

NOTE

MongoDB creates a database lazily — it only appears in 'show dbs' after the first document is insert
ed.

2.3 Creating Collections

```
# Implicit creation (on first insert)
db.customers.insertOne({ name: "Acme Corp", status: "active" })

# Explicit creation with options
db.createCollection("orders", {
    capped: false,
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["order_date", "customer_id", "total"],
            properties: {
                order_date: { bsonType: "date" },
                customer_id: { bsonType: "objectId" },
                total: { bsonType: "double", minimum: 0 },
                status: {
                    enum: ["pending", "shipped", "delivered", "cancelled"]
                }
            }
        }
    },
    validationLevel: "strict",
    validationAction: "error"
})

# List all collections
show collections
db.getCollectionNames()

# Get collection stats
db.orders.stats()

# Drop a collection
db.orders.drop()
```

2.4 Creating Capped Collections

Capped collections are fixed-size, circular buffers — ideal for logs and event streams.

```
db.createCollection("app_logs", {
    capped: true,
    size: 104857600,     // 100 MB max size
    max: 1000000         // Max 1 million documents
})
```

2.5 Indexes

2.5.1 Index Types and Creation

```
# Single field index
db.customers.createIndex({ email: 1 })              // Ascending
db.customers.createIndex({ created_at: -1 })         // Descending
```

```
# Compound index
db.orders.createIndex({ customer_id: 1, order_date: -1 })

# Unique index
db.customers.createIndex({ email: 1 }, { unique: true })

# Partial index (index only documents matching a filter)
db.orders.createIndex(
    { status: 1 },
    { partialFilterExpression: { status: "pending" } }
)

# TTL index (auto-expire documents)
db.sessions.createIndex(
    { created_at: 1 },
    { expireAfterSeconds: 3600 }   // Expire after 1 hour
)

# Text index (full-text search)
db.articles.createIndex({ title: "text", body: "text" })

# Wildcard index (index all fields in subdocuments)
db.products.createIndex({ "attributes.$**": 1 })

# Hashed index (for hash-based sharding)
db.users.createIndex({ user_id: "hashed" })
```

2.5.2 Index Management

```
# List all indexes on a collection
db.orders.getIndexes()

# Get index usage statistics
db.orders.aggregate([{ $indexStats: {} }])

# Drop a specific index
db.orders.dropIndex("status_1")

# Drop all indexes (except _id)
db.orders.dropIndexes()

# Rebuild indexes
db.orders.reIndex()

# Build index in background (default in 4.2+)
db.orders.createIndex(
    { total: 1 },
    { name: "idx_total", background: true }
)

# Hide an index (test performance impact without dropping)
db.orders.hideIndex("idx_total")
```

```
db.orders.unhideIndex("idx_total")
```

2.6 Views

```
# Create a view — acts like a stored aggregation
db.createView("active_customers", "customers", [
    { $match: { status: "active" } },
    { $project: { name: 1, email: 1, created_at: 1 } }
])

# Query the view like a normal collection
db.active_customers.find()

# Create a view with a $lookup (JOIN)
db.createView("order_details", "orders", [
    { $lookup: {
        from: "customers",
        localField: "customer_id",
        foreignField: "_id",
        as: "customer"
    }},
    { $unwind: "$customer" },
    { $project: {
        order_date: 1,
        total: 1,
        status: 1,
        "customer.name": 1
    }}
])

# Drop a view
db.active_customers.drop()
```

2.7 Schema Validation (Modify Existing Collection)

```
db.runCommand({
    collMod: "customers",
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["name", "email"],
            properties: {
                name:  { bsonType: "string", minLength: 1 },
                email: { bsonType: "string", pattern: "^.+@.+\\..+$" },
                phone: { bsonType: "string" },
                address: {
                    bsonType: "object",
                    properties: {
                        street: { bsonType: "string" },
                        city:   { bsonType: "string" },
```

```
                        zip:    { bsonType: "string" }
                    }
                }
            }
        }
    },
    validationLevel: "moderate",   // strict | moderate | off
    validationAction: "warn"       // error | warn
})
```

3 Data Import, Export, and ETL Operations

3.1 mongoimport — Load Data from Flat Files

3.1.1 Import JSON

```
# Import a JSON array file
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=customers \
    --file=/apps/mongodb/staging/customers.json \
    --jsonArray

# Import line-delimited JSON (one document per line)
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=events \
    --file=/apps/mongodb/staging/events.jsonl \
    --type=json

# Import with upsert (update existing, insert new)
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=customers \
    --file=/apps/mongodb/staging/customers_update.json \
    --jsonArray \
    --mode=upsert \
    --upsertFields=email
```

3.1.2 Import CSV / TSV

```
# Import CSV with header row
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=products \
    --type=csv \
    --headerline \
    --file=/apps/mongodb/staging/products.csv

# Import CSV with explicit field names and types
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=sales \
    --type=csv \
    --fields="date.date(),product_id.string(),qty.int32(),price.double()" \
    --file=/apps/mongodb/staging/sales.csv

# Import TSV
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=logs \
    --type=tsv \
    --headerline \
    --file=/apps/mongodb/staging/access_log.tsv
```

```
# Drop collection before import (clean load)
mongoimport --uri="mongodb://localhost:27017/appdb" \
    --collection=products \
    --drop \
    --type=csv \
    --headerline \
    --file=/apps/mongodb/staging/products.csv
```

3.2 mongoexport — Export Data to Flat Files

```
# Export entire collection to JSON
mongoexport --uri="mongodb://localhost:27017/appdb" \
    --collection=customers \
    --out=/apps/mongodb/staging/customers_export.json \
    --jsonArray --pretty

# Export specific fields
mongoexport --uri="mongodb://localhost:27017/appdb" \
    --collection=customers \
    --fields=name,email,status \
    --out=/apps/mongodb/staging/customers_partial.json

# Export with query filter
mongoexport --uri="mongodb://localhost:27017/appdb" \
    --collection=orders \
    --query='{"status":"shipped","total":{"$gte":1000}}' \
    --out=/apps/mongodb/staging/large_shipped_orders.json

# Export to CSV
mongoexport --uri="mongodb://localhost:27017/appdb" \
    --collection=customers \
    --type=csv \
    --fields=name,email,phone,status \
    --out=/apps/mongodb/staging/customers.csv
```

3.3 Bulk Insert from mongosh

```
// Bulk insert with ordered=false for maximum throughput
db.products.insertMany([
    { sku: "A001", name: "Widget", price: 9.99, qty: 100 },
    { sku: "A002", name: "Gadget", price: 19.99, qty: 50 },
    { sku: "A003", name: "Doohickey", price: 4.99, qty: 500 }
], { ordered: false })

// Bulk operations with bulkWrite (mixed insert/update/delete)
db.products.bulkWrite([
    { insertOne: { document: { sku: "B001", name: "New Item", price: 29.99 }}},
    { updateOne: {
        filter: { sku: "A001" },
        update: { $set: { price: 11.99 }, $inc: { qty: 50 }}
```

```
    }},
    { deleteOne: { filter: { sku: "A003" }}}
], { ordered: false })
```

3.4 BSON Dump and Restore (Binary Format)

For large-scale data movement between MongoDB instances, use mongodump/mongorestore (covered in Section 4) which operates on BSON — the native binary format. It is significantly faster than JSON-based import/export.

4 Backup and Restore — Full, Incremental, Point-in-Time

4.1 Backup Strategy Overview

Backup Type	Tool / Method	Scope	Use Case
Full Logical	mongodump	All DBs or specific DB/collection	Small–medium databases, cross-version migration
Full Filesystem	LVM/ZFS snapshot, cp	Entire data directory	Large databases, fastest full backup
Incremental (Oplog)	mongodump --oplog + oplog replay	Changes since last backup	Reduce backup window for large DBs
Point-in-Time (PITR)	Oplog replay to timestamp	Restore to exact second	Disaster recovery, accidental deletes
Cloud / Atlas	MongoDB Atlas continuous backup	Automated snapshots + PITR	Managed environments

NOTE

MongoDB does not have native 'differential' or 'delta' backups like RDBMS. The closest equivalent is oplog-based incremental backup, which captures only the operations since the last full backup.

4.2 Full Logical Backup with mongodump

```
# Full backup — all databases
mongodump --uri="mongodb://localhost:27017" \
    --out=/apps/mongodb/backups/full_$(date +%Y%m%d_%H%M%S)

# Backup a specific database
mongodump --uri="mongodb://localhost:27017/appdb" \
    --out=/apps/mongodb/backups/appdb_$(date +%Y%m%d)

# Backup a specific collection
mongodump --uri="mongodb://localhost:27017/appdb" \
    --collection=orders \
    --out=/apps/mongodb/backups/orders_$(date +%Y%m%d)

# Backup with compression (gzip)
mongodump --uri="mongodb://localhost:27017/appdb" \
    --gzip \
    --out=/apps/mongodb/backups/appdb_gz_$(date +%Y%m%d)

# Backup to a single archive file
mongodump --uri="mongodb://localhost:27017/appdb" \
    --archive=/apps/mongodb/backups/appdb_$(date +%Y%m%d).archive \
```

```
    --gzip

# Backup with oplog (for consistent point-in-time snapshot on replica set)
mongodump --uri="mongodb://localhost:27017" \
    --oplog \
    --gzip \
    --out=/apps/mongodb/backups/full_oplog_$(date +%Y%m%d)

# Backup with authentication
mongodump --uri="mongodb://admin:password@localhost:27017/?authSource=admin" \
    --out=/apps/mongodb/backups/full_$(date +%Y%m%d)
```

4.3 Full Restore with mongorestore

```
# Restore all databases from a full backup
mongorestore --uri="mongodb://localhost:27017" \
    /apps/mongodb/backups/full_20250101_120000/

# Restore a specific database
mongorestore --uri="mongodb://localhost:27017" \
    --nsInclude="appdb.*" \
    /apps/mongodb/backups/full_20250101_120000/

# Restore a single collection
mongorestore --uri="mongodb://localhost:27017" \
    --nsInclude="appdb.orders" \
    /apps/mongodb/backups/full_20250101_120000/

# Restore and drop existing collections first
mongorestore --uri="mongodb://localhost:27017" \
    --drop \
    /apps/mongodb/backups/full_20250101_120000/

# Restore from compressed backup
mongorestore --uri="mongodb://localhost:27017" \
    --gzip \
    /apps/mongodb/backups/appdb_gz_20250101/

# Restore from archive file
mongorestore --uri="mongodb://localhost:27017" \
    --archive=/apps/mongodb/backups/appdb_20250101.archive \
    --gzip

# Restore with oplog replay (point-in-time consistency)
mongorestore --uri="mongodb://localhost:27017" \
    --oplogReplay \
    /apps/mongodb/backups/full_oplog_20250101/
```

4.4 Granular Database-Level Backup and Restore

These operations target a specific database — backing up all its collections and indexes as a unit, and restoring them to the same or a different database.

4.4.1 Backup a Single Database

```
# Backup database "appdb" — all collections, indexes, and metadata
mongodump --uri="mongodb://localhost:27017" \
    --db=appdb \
    --out=/apps/mongodb/backups/db_appdb_$(date +%Y%m%d)

# With compression
mongodump --db=appdb --gzip \
    --out=/apps/mongodb/backups/db_appdb_gz_$(date +%Y%m%d)

# To a single archive file
mongodump --db=appdb --gzip \
    --archive=/apps/mongodb/backups/appdb_$(date +%Y%m%d).archive

# With authentication
mongodump -u admin -p --authenticationDatabase admin \
    --db=appdb \
    --out=/apps/mongodb/backups/db_appdb_$(date +%Y%m%d)

# Backup directory structure produced:
#   /apps/mongodb/backups/db_appdb_20250327/
#   ■■■ appdb/
#       ■■■ customers.bson          (data)
#       ■■■ customers.metadata.json (indexes, options, validation)
#       ■■■ orders.bson
#       ■■■ orders.metadata.json
#       ■■■ products.bson
#       ■■■ products.metadata.json
```

4.4.2 Restore a Single Database

```
# Restore database "appdb" from backup (preserves existing data)
mongorestore --uri="mongodb://localhost:27017" \
    --db=appdb \
    /apps/mongodb/backups/db_appdb_20250327/appdb/

# Restore and DROP existing collections first (clean restore)
mongorestore --db=appdb --drop \
    /apps/mongodb/backups/db_appdb_20250327/appdb/

# Restore from compressed backup
mongorestore --db=appdb --drop --gzip \
    /apps/mongodb/backups/db_appdb_gz_20250327/appdb/

# Restore from archive
mongorestore --db=appdb --drop --gzip \
    --archive=/apps/mongodb/backups/appdb_20250327.archive
```

4.4.3 Restore a Database Under a Different Name (Rename)

```
# Backup "appdb" and restore it as "appdb_staging"
# (useful for creating test/staging copies from production)

# Using --nsFrom / --nsTo (namespace remapping)
mongorestore --nsFrom="appdb.*" --nsTo="appdb_staging.*" \
    /apps/mongodb/backups/db_appdb_20250327/

# From an archive file with namespace remapping
mongorestore --gzip \
    --archive=/apps/mongodb/backups/appdb_20250327.archive \
    --nsFrom="appdb.*" --nsTo="appdb_test.*"

# Verify the restored database
mongosh --eval 'use appdb_staging; show collections; db.stats()'
```

4.4.4 Backup and Restore Multiple Databases (Selective)

```
# Backup only specific databases using --nsInclude
mongodump --uri="mongodb://localhost:27017" \
    --nsInclude="appdb.*" --nsInclude="logsdb.*" \
    --out=/apps/mongodb/backups/selected_dbs_$(date +%Y%m%d)

# Backup all databases EXCEPT specific ones using --nsExclude
mongodump --uri="mongodb://localhost:27017" \
    --nsExclude="test.*" --nsExclude="tempdb.*" \
    --out=/apps/mongodb/backups/production_dbs_$(date +%Y%m%d)

# Restore only specific databases from a full backup
mongorestore --nsInclude="appdb.*" \
    /apps/mongodb/backups/full_20250327/

# Restore everything except certain databases
mongorestore --nsExclude="test.*" --nsExclude="tempdb.*" \
    /apps/mongodb/backups/full_20250327/
```

4.5 Granular Collection-Level (Table-Level) Backup and Restore

These operations target individual collections — the MongoDB equivalent of table-level backup and restore in RDBMS. This is the most granular level of backup/restore available.

4.5.1 Backup a Single Collection

```
# Backup collection "orders" from database "appdb"
mongodump --uri="mongodb://localhost:27017" \
    --db=appdb \
    --collection=orders \
    --out=/apps/mongodb/backups/coll_orders_$(date +%Y%m%d)

# With compression
```

```
mongodump --db=appdb --collection=orders --gzip \
    --out=/apps/mongodb/backups/coll_orders_gz_$(date +%Y%m%d)

# To an archive file (single collection)
mongodump --db=appdb --collection=orders --gzip \
    --archive=/apps/mongodb/backups/orders_$(date +%Y%m%d).archive

# Backup a collection with a query filter (partial backup)
# Only backup orders from 2025
mongodump --db=appdb --collection=orders \
    --query='{"order_date":{"$gte":{"$date":"2025-01-01T00:00:00Z"}}}' \
    --out=/apps/mongodb/backups/orders_2025_$(date +%Y%m%d)

# Backup directory structure produced:
#   /apps/mongodb/backups/coll_orders_20250327/
#   ■■■ appdb/
#       ■■■ orders.bson              (document data in BSON)
#       ■■■ orders.metadata.json     (indexes, validation rules)
```

4.5.2 Restore a Single Collection

```
# Restore collection "orders" to database "appdb"
mongorestore --uri="mongodb://localhost:27017" \
    --db=appdb \
    --collection=orders \
    /apps/mongodb/backups/coll_orders_20250327/appdb/orders.bson

# Drop existing collection before restoring (clean restore)
mongorestore --db=appdb --collection=orders --drop \
    /apps/mongodb/backups/coll_orders_20250327/appdb/orders.bson

# Restore from compressed backup
mongorestore --db=appdb --collection=orders --drop --gzip \
    /apps/mongodb/backups/coll_orders_gz_20250327/appdb/orders.bson

# Restore from archive
mongorestore --db=appdb --collection=orders --drop --gzip \
    --archive=/apps/mongodb/backups/orders_20250327.archive

# Restore with --noIndexRestore (data only, skip index recreation)
mongorestore --db=appdb --collection=orders --drop \
    --noIndexRestore \
    /apps/mongodb/backups/coll_orders_20250327/appdb/orders.bson
```

4.5.3 Restore a Collection Under a Different Name (Rename)

```
# Restore "orders" as "orders_backup" in the same database
mongorestore --db=appdb --collection=orders_backup \
    /apps/mongodb/backups/coll_orders_20250327/appdb/orders.bson

# Restore "orders" from "appdb" into a completely different database
```

```
mongorestore --db=staging_db --collection=orders \
    /apps/mongodb/backups/coll_orders_20250327/appdb/orders.bson

# Restore using namespace remapping (from archive)
mongorestore --gzip \
    --archive=/apps/mongodb/backups/orders_20250327.archive \
    --nsFrom="appdb.orders" --nsTo="staging_db.orders_copy"
```

4.5.4 Backup and Restore Multiple Specific Collections

```
# Backup multiple specific collections using --nsInclude
mongodump --uri="mongodb://localhost:27017" \
    --nsInclude="appdb.orders" \
    --nsInclude="appdb.customers" \
    --nsInclude="appdb.products" \
    --out=/apps/mongodb/backups/selected_colls_$(date +%Y%m%d)

# Backup all collections EXCEPT specific ones
mongodump --db=appdb \
    --nsExclude="appdb.app_logs" \
    --nsExclude="appdb.sessions" \
    --out=/apps/mongodb/backups/appdb_no_logs_$(date +%Y%m%d)

# Restore only specific collections from a database backup
mongorestore --nsInclude="appdb.orders" --nsInclude="appdb.customers" \
    --drop \
    /apps/mongodb/backups/full_20250327/
```

4.5.5 Cross-Server Collection Copy (Remote Backup and Restore)

```
# Backup a collection from a remote server
mongodump --host=prod-server.internal --port=27017 \
    -u backup_user -p --authenticationDatabase admin \
    --db=appdb --collection=orders --gzip \
    --out=/apps/mongodb/backups/remote_orders_$(date +%Y%m%d)

# Pipe directly from one server to another (no intermediate files)
mongodump --host=prod-server.internal --port=27017 \
    -u backup_user -p --authenticationDatabase admin \
    --db=appdb --collection=orders \
    --archive --gzip | \
mongorestore --host=staging-server.internal --port=27017 \
    -u admin -p --authenticationDatabase admin \
    --archive --gzip \
    --nsFrom="appdb.orders" --nsTo="staging_db.orders" --drop

# Pipe an entire database between servers
mongodump --host=prod-server.internal \
    --db=appdb --archive --gzip | \
mongorestore --host=staging-server.internal \
    --archive --gzip --drop
```

4.6 Collection-Level Backup from Inside mongosh

In addition to the mongodump/mongorestore CLI tools, you can perform lightweight backup and restore operations directly from the mongosh shell using aggregation and JavaScript.

```
// Copy a collection within the same database
db.orders.aggregate([{ $out: "orders_backup" }])

// Copy a collection to a different database (MongoDB 4.4+)
db.orders.aggregate([
    { $merge: {
        into: { db: "backup_db", coll: "orders_snapshot" },
        whenMatched: "replace",
        whenNotMatched: "insert"
    }}
])

// Copy only matching documents (partial backup)
db.orders.aggregate([
    { $match: { status: "shipped", order_date: { $gte: ISODate("2025-01-01") }}},
    { $out: "shipped_orders_2025" }
])

// Clone entire database collections via script
var colls = db.getCollectionNames()
colls.forEach(function(c) {
    if (!c.startsWith("system.")) {
        print("Copying: " + c + " -> " + c + "_bak")
        db[c].aggregate([{ $out: c + "_bak" }])
    }
})

// Export a collection to JSON from mongosh (for small datasets)
var cursor = db.orders.find()
var docs = cursor.toArray()
// Then use mongoexport CLI for larger datasets
```

NOTE

The $out stage REPLACES the target collection entirely. Use $merge for incremental/upsert operations
.
$out can only write to the same database; $merge can write cross-database.

4.7 Backup and Restore Quick Reference Matrix

Scope	Backup Command	Restore Command
All databases	mongodump --out=/path/backup/	mongorestore /path/backup/
Single database	mongodump --db=mydb --out=/path/	mongorestore --db=mydb /path/mydb/
Single collection	mongodump --db=mydb --collection=coll --out=/path/	mongorestore --db=mydb --collection=coll /path/mydb/coll.bson

Filtered documents	mongodump --db=mydb --collection=coll --query='{"status":"active"}'	(restores whatever was dumped)
Rename DB on restore	(normal dump)	mongorestore --nsFrom="old.*" --nsTo="new.*"
Rename collection	(normal dump)	mongorestore --db=mydb --collection=new_name /path/old.bson
Compressed	mongodump --gzip --out=/path/	mongorestore --gzip /path/
Archive file	mongodump --archive=/path/file.archive --gzip	mongorestore --archive=/path/file.archive --gzip
Cross-server pipe	mongodump --host=src --archive | mongorestore --host=dst --archive	(single command as shown)
Drop before restore	(normal dump)	mongorestore --drop /path/
Data only (no indexes)	(normal dump)	mongorestore --noIndexRestore /path/
Collection copy (shell)	db.coll.aggregate([{$out:"coll_bak"}])	(creates target collection directly)

4.8 Filesystem Snapshot Backup (LVM)

```
# Step 1: Freeze writes (if not using journaling on a replica set)
mongosh --eval "db.fsyncLock()"

# Step 2: Take LVM snapshot
sudo lvcreate --size 50G --snapshot --name mongodb_snap \
    /dev/vg_apps/lv_mongodb_data

# Step 3: Unfreeze writes
mongosh --eval "db.fsyncUnlock()"

# Step 4: Mount snapshot and copy data
sudo mkdir -p /mnt/mongodb_snap
sudo mount /dev/vg_apps/mongodb_snap /mnt/mongodb_snap
sudo cp -a /mnt/mongodb_snap/* /apps/mongodb/backups/snap_$(date +%Y%m%d)/

# Step 5: Cleanup
sudo umount /mnt/mongodb_snap
sudo lvremove -f /dev/vg_apps/mongodb_snap
```

4.9 Incremental Backup via Oplog

MongoDB's oplog (operation log) is a capped collection on replica set members that records every write. You can use it for incremental backups by capturing oplog entries since the last backup.

```
# Step 1: Record the timestamp of the last full backup
LAST_TS=$(mongosh --quiet --eval '
    db.getSiblingDB("local").oplog.rs.find().sort({ts:-1}).limit(1)
    .next().ts
```

```
')
echo "Last backup timestamp: $LAST_TS"

# Step 2: Dump oplog entries since last backup
mongodump --uri="mongodb://localhost:27017/local" \
    --collection=oplog.rs \
    --query="{"ts":{"\$gt":Timestamp($LAST_TS)}}" \
    --out=/apps/mongodb/backups/oplog_incr_$(date +%Y%m%d)

# Step 3: To restore — first restore full backup, then replay oplog
mongorestore --oplogReplay \
    /apps/mongodb/backups/full_oplog_20250101/
```

4.10 Point-in-Time Recovery (PITR)

```
# Restore full backup then replay oplog up to a specific timestamp
mongorestore --uri="mongodb://localhost:27017" \
    --oplogReplay \
    --oplogLimit="1735689600:1" \
    --drop \
    /apps/mongodb/backups/full_oplog_20250101/

# The oplogLimit is a BSON Timestamp(seconds, increment)
# This replays the oplog up to but not including the specified timestamp
```

4.11 Automated Backup Script

```
#!/bin/bash
# /apps/mongodb/scripts/backup_full.sh
# Schedule via cron: 0 2 * * * /apps/mongodb/scripts/backup_full.sh

BACKUP_DIR="/apps/mongodb/backups"
DATE=$(date +%Y%m%d_%H%M%S)
RETENTION_DAYS=7
URI="mongodb://backup_user:password@localhost:27017/?authSource=admin"

echo "[$(date)] Starting full backup..."
mongodump --uri="$URI" --oplog --gzip \
    --out="$BACKUP_DIR/full_$DATE"

if [ $? -eq 0 ]; then
    echo "[$(date)] Backup completed successfully: full_$DATE"
else
    echo "[$(date)] ERROR: Backup failed!" >&2
    exit 1
fi

# Cleanup old backups
find "$BACKUP_DIR" -maxdepth 1 -name "full_*" -type d \
    -mtime +$RETENTION_DAYS -exec rm -rf {} \;
```

```
echo "[$(date)] Cleanup done. Retained last $RETENTION_DAYS days."
```

5 Troubleshooting — Locks, Monitoring, Active Operations

5.1 Monitor What's Running — currentOp

```
# Show ALL active operations
db.currentOp()

# Show only active (non-idle) operations
db.currentOp({ active: true })

# Show operations running longer than 10 seconds
db.currentOp({
    active: true,
    secs_running: { $gte: 10 }
})

# Show operations on a specific database
db.currentOp({
    active: true,
    ns: /^appdb\./
})

# Show only write operations (insert, update, delete)
db.currentOp({
    active: true,
    op: { $in: ["insert", "update", "remove"] }
})

# Show operations waiting for locks
db.currentOp({
    waitingForLock: true
})

# Show queries (not internal operations)
db.currentOp({
    active: true,
    op: "query",
    ns: { $ne: "" }
})
```

5.2 Detect and Analyze Locks

MongoDB uses document-level locking (WiredTiger engine). While true deadlocks are rare, long-running writes can cause contention.

```
# Show operations that are waiting for a lock
db.currentOp({ waitingForLock: true })

# Show lock information for active operations
```

```
db.currentOp({ active: true }).inprog.forEach(function(op) {
    if (op.locks || op.waitingForLock) {
        print("OpId:", op.opid,
              "| NS:", op.ns,
              "| Op:", op.op,
              "| Running:", op.secs_running, "sec",
              "| WaitingForLock:", op.waitingForLock || false)
    }
})

# Server-wide lock status
db.serverStatus().locks

# Global lock statistics
db.serverStatus().globalLock

# Check lock percentage (high values indicate contention)
var status = db.serverStatus()
var gl = status.globalLock
print("Current queue — readers:", gl.currentQueue.readers,
      "| writers:", gl.currentQueue.writers)
print("Active clients — readers:", gl.activeClients.readers,
      "| writers:", gl.activeClients.writers)
```

5.3 Kill / Terminate Operations

```
# Kill a specific operation by its opid
db.killOp(<opid>)

# Example: kill operation 12345
db.killOp(12345)

# Kill all operations running longer than 60 seconds
db.currentOp({ active: true, secs_running: { $gte: 60 } })
    .inprog.forEach(function(op) {
        print("Killing opid:", op.opid, "running for", op.secs_running, "sec")
        db.killOp(op.opid)
    })

# Kill all operations on a specific collection
db.currentOp({ active: true, ns: "appdb.orders" })
    .inprog.forEach(function(op) {
        db.killOp(op.opid)
    })

# Kill operations from a specific client/appName
db.currentOp({ active: true, appName: "myBatchJob" })
    .inprog.forEach(function(op) {
        db.killOp(op.opid)
    })
```

WARNING

db.killOp() sends a kill request — the operation may not terminate instantly.
Some operations (like index builds) can only be killed at certain checkpoints.

5.4 Connection Monitoring

```
# Current connections
db.serverStatus().connections
// { current: 45, available: 455, totalCreated: 1234 }

# Per-client connection info
db.currentOp(true).inprog.forEach(function(op) {
    print("Client:", op.client, "| AppName:", op.appName, "| Op:", op.op)
})

# Connection pool stats (from application driver)
db.serverStatus().network
```

5.5 Real-Time Monitoring with mongostat and mongotop

```
# mongostat — server-wide metrics every 2 seconds
mongostat --uri="mongodb://localhost:27017" 2

# Key columns to watch:
#   insert/query/update/delete — operations per second
#   getmore — cursor batches
#   res     — resident memory
#   qrw     — queued read|write (should be 0|0)
#   arw     — active read|write

# mongotop — per-collection read/write time every 5 seconds
mongotop --uri="mongodb://localhost:27017" 5
```

5.6 Log Analysis

```
# View recent slow operations from the log
grep "Slow query" /apps/mongodb/log/mongod.log | tail -20

# Filter for operations exceeding 1000ms
grep -E '"durationMillis":[0-9]{4,}' /apps/mongodb/log/mongod.log | tail -10

# Real-time log monitoring
tail -f /apps/mongodb/log/mongod.log | grep --line-buffered "COMMAND"

# Set log verbosity dynamically
db.setLogLevel(1, "query")       // Increase query logging
db.setLogLevel(0, "query")       // Reset to default
```

```
db.getLogComponents()             // View all log component levels
```

5.7 Health Check Commands

```
# Overall server status
db.serverStatus()

# Quick health ping
db.runCommand({ ping: 1 })

# Replication status (on replica set)
rs.status()
rs.printReplicationInfo()      // Oplog window
rs.printSecondaryReplicationInfo()  // Replication lag

# Storage stats for a database
db.stats()

# Collection-level stats
db.orders.stats()
db.orders.storageSize()
db.orders.totalIndexSize()
```

6 Performance Tuning — Slow Queries, Profiler, Indexes

6.1 The Database Profiler

The profiler captures detailed information about operations that exceed a threshold. This is the primary tool for identifying slow queries.

```
# Enable profiler — capture operations slower than 100ms
db.setProfilingLevel(1, { slowms: 100 })

# Profile ALL operations (use only briefly — high overhead!)
db.setProfilingLevel(2)

# Disable profiler
db.setProfilingLevel(0)

# Check current profiler status
db.getProfilingStatus()

# Query the system.profile collection for slow operations
db.system.profile.find({
    millis: { $gt: 200 }
}).sort({ ts: -1 }).limit(10).pretty()

# Find the slowest operations by namespace
db.system.profile.find().sort({ millis: -1 }).limit(5).pretty()

# Find queries doing collection scans (COLLSCAN = no index used)
db.system.profile.find({
    "planSummary": "COLLSCAN"
}).sort({ ts: -1 }).limit(10)
```

6.2 Explain Plans (Query Execution Analysis)

```
# Basic explain
db.orders.find({ status: "pending" }).explain()

# Detailed execution stats
db.orders.find({ status: "pending" }).explain("executionStats")

# Full explain with all candidate plans
db.orders.find({ status: "pending" }).explain("allPlansExecution")

# Key fields to analyze:
//   winningPlan.stage:
//     COLLSCAN  — full collection scan (BAD for large collections)
//     IXSCAN    — index scan (GOOD)
//     FETCH     — document fetch after index lookup
//     SORT      — in-memory sort (check if index can avoid it)
```

```
//
//   executionStats:
//     totalDocsExamined     — should be close to nReturned
//     totalKeysExamined     — index entries scanned
//     nReturned             — documents returned to client
//     executionTimeMillis   — query time
//
//   RATIO CHECK: totalDocsExamined / nReturned should be ~1.0
//   If this ratio is high, the query is scanning too many documents.

# Explain an aggregation pipeline
db.orders.explain("executionStats").aggregate([
    { $match: { status: "pending" } },
    { $group: { _id: "$customer_id", total: { $sum: "$total" }}}
])
```

6.3 Index Optimization Strategies

6.3.1 The ESR Rule (Equality → Sort → Range)

When designing compound indexes, place fields in this order for optimal performance: Equality match fields first, then Sort fields, then Range filter fields.

```
# Query pattern:
#   db.orders.find({ status: "pending", total: { $gte: 100 } })
#             .sort({ order_date: -1 })
#
# Optimal compound index (ESR):
db.orders.createIndex({
    status: 1,         // Equality
    order_date: -1,    // Sort
    total: 1           // Range
})
```

6.3.2 Covered Queries

A covered query is satisfied entirely from the index without fetching documents — the fastest possible query.

```
# Index covers the query if all queried and projected fields are in the index
db.orders.createIndex({ customer_id: 1, status: 1, total: 1 })

# This query is covered (no FETCH stage in explain)
db.orders.find(
    { customer_id: ObjectId("..."), status: "shipped" },
    { total: 1, _id: 0 }    // Project only indexed fields, exclude _id
)
```

6.3.3 Identifying Unused Indexes

```
// Get index usage stats — look for indexes with 0 accesses
db.orders.aggregate([{ $indexStats: {} }]).forEach(function(idx) {
    print(idx.name, "| accesses:", idx.accesses.ops,
          "| since:", idx.accesses.since)
})

// Drop unused indexes to save disk space and reduce write overhead
```

6.4 WiredTiger Cache Tuning

```
# View current cache usage
db.serverStatus().wiredTiger.cache

# Key metrics:
//   "bytes currently in the cache"
//   "maximum bytes configured"
//   "tracked dirty bytes in the cache"
//   "pages evicted"

# Adjust cache size (in mongod.conf)
# Rule of thumb: 50% of RAM minus 1 GB, or at least 256 MB
# Example for 16 GB RAM server:
#   storage.wiredTiger.engineConfig.cacheSizeGB: 7

# Dynamic adjustment (no restart required)
db.adminCommand({
    setParameter: 1,
    wiredTigerCacheSizeGB: 7
})
```

6.5 Connection Pool Tuning

```
# Check current connection utilization
db.serverStatus().connections

# If connections approach maxIncomingConnections (default 65536):
# 1. Check for connection leaks in application code
# 2. Tune application connection pool (maxPoolSize, minPoolSize)
# 3. Adjust mongod max connections if needed:
#    net.maxIncomingConnections: 500    # in mongod.conf

# Monitor connection churn
db.serverStatus().network
```

6.6 Aggregation Pipeline Optimization

```
# Best practices:
# 1. Place $match and $limit as early as possible
```

```
# 2. Use $project early to reduce document size in pipeline
# 3. Use allowDiskUse for large sorts/groups

db.orders.aggregate([
    { $match: { status: "shipped", order_date: { $gte: ISODate("2025-01-01") }}},
    { $project: { customer_id: 1, total: 1, order_date: 1 }},
    { $group: {
        _id: "$customer_id",
        order_count: { $sum: 1 },
        total_revenue: { $sum: "$total" }
    }},
    { $sort: { total_revenue: -1 }},
    { $limit: 100 }
], { allowDiskUse: true })
```

6.7 Overall Database Slow — Diagnostic Checklist

Symptom	Check	Resolution
High CPU	db.currentOp() for long queries; mongostat for op rates	Add missing indexes; optimize queries; scale out reads
High memory / OOM	serverStatus().wiredTiger.cache; free -m	Tune cacheSizeGB; check for large in-memory sorts
Disk I/O saturation	iostat -xz 2; mongotop	Move to faster storage (SSD/NVMe); reduce COLLSCAN queries
Connection exhaustion	serverStatus().connections	Fix connection leaks; tune pool size; increase maxIncomingConnections
Replication lag	rs.printSecondaryReplicationInfo()	Reduce write load; increase oplog size; check network bandwidth
Lock contention	serverStatus().globalLock; currentOp waitingForLock	Optimize write patterns; reduce long-running transactions

7 Security — Authentication, Authorization, Encryption, Auditing

7.1 Enable Authentication

```
# Step 1: Create the admin user BEFORE enabling auth
mongosh
use admin
db.createUser({
    user: "admin",
    pwd: passwordPrompt(),    // Prompts securely; or use "password" for scripts
    roles: [
        { role: "userAdminAnyDatabase", db: "admin" },
        { role: "readWriteAnyDatabase", db: "admin" },
        { role: "clusterAdmin", db: "admin" }
    ]
})

# Step 2: Enable authorization in mongod.conf
# security:
#   authorization: enabled

# Step 3: Restart mongod
sudo systemctl restart mongod

# Step 4: Connect with authentication
mongosh -u admin -p --authenticationDatabase admin
```

7.2 Role-Based Access Control (RBAC)

```
# Create an application-specific user
use appdb
db.createUser({
    user: "app_user",
    pwd: "securePassword123!",
    roles: [
        { role: "readWrite", db: "appdb" }
    ]
})

# Create a read-only reporting user
db.createUser({
    user: "report_user",
    pwd: "reportPass456!",
    roles: [
        { role: "read", db: "appdb" }
    ]
})

# Create a backup-specific user
```

```
use admin
db.createUser({
    user: "backup_user",
    pwd: "backupPass789!",
    roles: [
        { role: "backup", db: "admin" },
        { role: "restore", db: "admin" }
    ]
})

# Built-in roles reference:
//   read              — Read-only on a database
//   readWrite         — Read and write on a database
//   dbAdmin           — Schema management, indexing, stats
//   userAdmin         — Create/modify users and roles
//   clusterAdmin      — Replica set and sharding management
//   backup / restore  — mongodump / mongorestore privileges
//   root              — Superuser (use sparingly)

# Create a custom role
use appdb
db.createRole({
    role: "orderManager",
    privileges: [
        {
            resource: { db: "appdb", collection: "orders" },
            actions: ["find", "update", "insert"]
        },
        {
            resource: { db: "appdb", collection: "customers" },
            actions: ["find"]
        }
    ],
    roles: []
})

# Assign custom role to user
db.createUser({
    user: "order_app",
    pwd: "orderPass!",
    roles: [{ role: "orderManager", db: "appdb" }]
})
```

7.3 TLS/SSL Encryption

```
# mongod.conf — TLS configuration
# net:
#   tls:
#     mode: requireTLS
#     certificateKeyFile: /apps/mongodb/ssl/server.pem
#     CAFile: /apps/mongodb/ssl/ca.pem
```

```
# Generate self-signed certs for testing
openssl req -newkey rsa:4096 -nodes -x509 -days 365 \
    -keyout /apps/mongodb/ssl/server.key \
    -out /apps/mongodb/ssl/server.crt \
    -subj "/CN=mongodb-server"

# Combine key and cert into PEM
cat /apps/mongodb/ssl/server.key /apps/mongodb/ssl/server.crt \
    > /apps/mongodb/ssl/server.pem

sudo chown mongod:mongod /apps/mongodb/ssl/*
sudo chmod 600 /apps/mongodb/ssl/server.pem

# Connect with TLS
mongosh --tls --tlsCAFile /apps/mongodb/ssl/ca.pem \
    --host mongodb-server:27017
```

7.4 Encryption at Rest

```
# mongod.conf — encryption at rest (Enterprise only)
# security:
#   enableEncryption: true
#   encryptionKeyFile: /apps/mongodb/keyfile/encryption-key

# For Community Edition, use LUKS disk-level encryption:
sudo cryptsetup luksFormat /dev/sdb
sudo cryptsetup luksOpen /dev/sdb mongodb_encrypted
sudo mkfs.xfs /dev/mapper/mongodb_encrypted
sudo mount /dev/mapper/mongodb_encrypted /apps/mongodb/data
```

7.5 Audit Logging (Enterprise)

```
# mongod.conf — audit configuration
# auditLog:
#   destination: file
#   format: JSON
#   path: /apps/mongodb/log/audit.json
#   filter: '{ atype: { $in: ["authenticate","createUser","dropDatabase"]}}'
```

7.6 Network Hardening

```
# Restrict bind IP to specific interfaces
# net:
#   bindIp: 127.0.0.1,10.10.1.50    # localhost + private network IP

# Use firewall rules to restrict access
sudo firewall-cmd --permanent --add-rich-rule='
    rule family="ipv4" source address="10.10.1.0/24"
```

```
    port protocol="tcp" port="27017" accept'
sudo firewall-cmd --permanent --remove-port=27017/tcp
sudo firewall-cmd --reload
```

8 Replication — Replica Sets, Failover, Monitoring

8.1 Replica Set Architecture

A MongoDB replica set consists of a minimum of 3 members: one primary and two secondaries (or one secondary and one arbiter). The primary receives all writes; secondaries replicate from the oplog.

8.2 Initialize a Replica Set

```
# On each member, set replSetName in mongod.conf:
# replication:
#   replSetName: rs0
#   oplogSizeMB: 2048

# On the designated primary, connect and initiate:
mongosh
rs.initiate({
    _id: "rs0",
    members: [
        { _id: 0, host: "mongo1.internal:27017", priority: 10 },
        { _id: 1, host: "mongo2.internal:27017", priority: 5 },
        { _id: 2, host: "mongo3.internal:27017", priority: 1 }
    ]
})

# Check replica set status
rs.status()
rs.conf()
```

8.3 Keyfile Authentication for Replica Set Members

```
# Generate keyfile (all members must share the same keyfile)
openssl rand -base64 756 > /apps/mongodb/keyfile/mongo-keyfile
chmod 400 /apps/mongodb/keyfile/mongo-keyfile
chown mongod:mongod /apps/mongodb/keyfile/mongo-keyfile

# Copy to all replica set members:
scp /apps/mongodb/keyfile/mongo-keyfile mongo2:/apps/mongodb/keyfile/
scp /apps/mongodb/keyfile/mongo-keyfile mongo3:/apps/mongodb/keyfile/

# Enable in mongod.conf on ALL members:
# security:
#   keyFile: /apps/mongodb/keyfile/mongo-keyfile
#   authorization: enabled
```

8.4 Add/Remove Replica Set Members

```
# Add a new secondary
rs.add("mongo4.internal:27017")

# Add an arbiter (votes but holds no data)
rs.addArb("mongo-arb.internal:27017")

# Remove a member
rs.remove("mongo4.internal:27017")

# Reconfigure member priority
cfg = rs.conf()
cfg.members[1].priority = 0     // Cannot become primary
cfg.members[1].hidden = true    // Hidden from application reads
cfg.members[1].votes = 0
rs.reconfig(cfg)
```

8.5 Monitoring Replication Health

```
# Replication lag
rs.printSecondaryReplicationInfo()
# Shows: source, syncedTo, and how many seconds behind the primary

# Oplog window (how far back the oplog goes)
rs.printReplicationInfo()
# Shows: oplog size, first/last event time, oplog window hours

# Detailed member states
rs.status().members.forEach(function(m) {
    print(m.name, "| state:", m.stateStr,
          "| optimeDate:", m.optimeDate,
          "| lag:", m.lag || 0, "sec")
})

# Force a secondary to resync (if too far behind)
# On the secondary:
db.adminCommand({ resync: true })
```

8.6 Manual Failover and Step-Down

```
# Step down the current primary (triggers election)
rs.stepDown(120)    // Refuses to be primary for 120 seconds

# Force a specific member to become primary
cfg = rs.conf()
cfg.members[2].priority = 100
rs.reconfig(cfg)
// Then step down the current primary

# Freeze a secondary (prevent it from seeking election)
rs.freeze(300)      // Frozen for 300 seconds
```

9 Sharding — Horizontal Scaling, Shard Key Selection

9.1 Sharding Architecture Overview

Sharding distributes data across multiple machines. The architecture consists of three components: shard servers (each a replica set holding a data subset), config servers (a replica set storing cluster metadata), and mongos routers (query routers that direct operations to the correct shard).

9.2 Deploy a Sharded Cluster

```
# Step 1: Start config server replica set (3 members)
# mongod.conf for config servers:
# sharding:
#   clusterRole: configsvr
# replication:
#   replSetName: configRS

# Step 2: Start shard replica sets
# mongod.conf for each shard:
# sharding:
#   clusterRole: shardsvr
# replication:
#   replSetName: shard1RS   (shard2RS, shard3RS, etc.)

# Step 3: Start mongos router
mongos --configdb configRS/cfg1:27019,cfg2:27019,cfg3:27019 \
    --bind_ip 0.0.0.0 --port 27017

# Step 4: Connect to mongos and add shards
mongosh --port 27017
sh.addShard("shard1RS/shard1a:27018,shard1b:27018")
sh.addShard("shard2RS/shard2a:27018,shard2b:27018")

# Step 5: Enable sharding on a database and shard a collection
sh.enableSharding("appdb")
sh.shardCollection("appdb.orders", { customer_id: "hashed" })
```

9.3 Shard Key Selection Guidelines

Strategy	Example	Pros	Cons
Hashed	{ user_id: "hashed" }	Even distribution	No range queries on shard key
Range	{ created_at: 1 }	Good for range scans	Hot spots on recent data
Compound	{ region: 1, created_at: 1 }	Balanced distribution + range queries	More complex to design

9.4 Monitor Sharding

```
# Cluster status
sh.status()

# Balancer status
sh.isBalancerRunning()
sh.getBalancerState()

# Chunk distribution per shard
db.orders.getShardDistribution()

# Enable/disable balancer
sh.stopBalancer()
sh.startBalancer()
```

10 Automation — Cron Jobs, Log Rotation, Housekeeping

10.1 Log Rotation with logrotate

```
# /etc/logrotate.d/mongodb
/apps/mongodb/log/mongod.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    sharedscripts
    postrotate
        /bin/kill -SIGUSR1 $(cat /apps/mongodb/data/mongod.lock 2>/dev/null) \
            2>/dev/null || true
    endscript
}
```

10.2 Cron Job Examples

```
# /etc/cron.d/mongodb-maintenance

# Full backup every day at 2 AM
0 2 * * * mongod /apps/mongodb/scripts/backup_full.sh >> /apps/mongodb/log/backup.log 2>&1

# Compact collections every Sunday at 4 AM
0 4 * * 0 mongod mongosh --eval 'db.getSiblingDB("appdb").orders.compact()' >> /apps/mongodb/log/maintenanc

# Remove expired sessions every hour
0 * * * * mongod mongosh appdb --eval 'db.sessions.deleteMany({expires:{$lt:new Date()}})' >> /apps/mongodb

# Health check every 5 minutes (alert on failure)
*/5 * * * * mongod mongosh --eval 'db.runCommand({ping:1})' || echo "MongoDB DOWN" | mail -s "ALERT" dba@co
```

10.3 Monitoring Script

```
#!/bin/bash
# /apps/mongodb/scripts/health_check.sh

URI="mongodb://monitor_user:password@localhost:27017/?authSource=admin"

# Check if mongod is running
if ! pgrep -x mongod > /dev/null; then
    echo "CRITICAL: mongod process not running!"
    exit 2
```

```
fi

# Check connectivity
if ! mongosh "$URI" --eval "db.runCommand({ping:1})" --quiet > /dev/null 2>&1; then
    echo "CRITICAL: Cannot connect to MongoDB!"
    exit 2
fi

# Check replication lag (if replica set)
LAG=$(mongosh "$URI" --eval '
    var status = rs.status();
    if (status.ok) {
        var primary = status.members.filter(m => m.state === 1)[0];
        var maxLag = 0;
        status.members.forEach(function(m) {
            if (m.state === 2) {
                var lag = (primary.optimeDate - m.optimeDate) / 1000;
                if (lag > maxLag) maxLag = lag;
            }
        });
        print(maxLag);
    } else { print(0); }
' --quiet 2>/dev/null)

if [ "$LAG" -gt 60 ]; then
    echo "WARNING: Replication lag is ${LAG} seconds"
    exit 1
fi

# Check disk usage
USAGE=$(df /apps/mongodb/data --output=pcent | tail -1 | tr -d '% ')
if [ "$USAGE" -gt 85 ]; then
    echo "WARNING: Data disk usage at ${USAGE}%"
    exit 1
fi

echo "OK: MongoDB healthy | lag=${LAG}s disk=${USAGE}%"
exit 0
```

11 Upgrading and Patching MongoDB

11.1 Pre-Upgrade Checklist

Step	Command / Action	Purpose
1. Check current version	mongod --version; rpm -qa | grep mongo | sort	Document baseline before changes
2. Review release notes	Visit MongoDB release notes page	Identify breaking changes, deprecations
3. Check FCV	db.adminCommand({getParameter:1, featureCompatibilityVersion:1})	Must match current major version
4. Full backup	mongodump --oplog --gzip --out=/apps/mongodb/backups/pre_upgrade	Rollback safety net
5. Test in staging	Upgrade a non-production instance first	Validate application compatibility
6. Plan maintenance window	Notify stakeholders	Coordinate downtime if standalone

11.2 Minor Version Upgrade via DNF/YUM (e.g., 7.0.8 → 7.0.12)

Minor (patch) upgrades within the same major.minor series are the most common maintenance activity. These are binary-compatible and do not require FCV changes.

```
# Step 1: Check current version and installed RPMs
mongod --version
rpm -qa | grep mongo | sort

# Step 2: Full backup
mongodump --oplog --gzip \
    --out=/apps/mongodb/backups/pre_patch_$(date +%Y%m%d)

# Step 3: Check what updates are available
sudo dnf check-update mongodb-org*

# Step 4: For STANDALONE — stop, upgrade, start
sudo systemctl stop mongod
sudo dnf update -y mongodb-org
sudo systemctl start mongod
mongod --version    # Confirm new version

# Step 4 (ALT): For REPLICA SET — rolling upgrade
# Upgrade secondaries first, one at a time:
#   On each SECONDARY:
sudo systemctl stop mongod
sudo dnf update -y mongodb-org
```

```
sudo systemctl start mongod
#   Wait for secondary to catch up:
mongosh --eval "rs.status()"

#   Then step down and upgrade the primary:
mongosh --eval "rs.stepDown()"
sudo systemctl stop mongod
sudo dnf update -y mongodb-org
sudo systemctl start mongod

# Step 5: Verify all RPMs updated
rpm -qa | grep mongo | sort
```

11.3 Minor Version Upgrade via Manual RPM (Air-Gapped / Offline)

When your private cloud VMs cannot reach the MongoDB repository, download the new RPMs externally and apply them locally.

```
# Step 1: On an internet-connected machine, download new RPMs
MONGO_VER="7.0.12"
RHEL_VER="9"
BASE="https://repo.mongodb.org/yum/redhat/${RHEL_VER}/mongodb-org/7.0/x86_64/RPMS"
mkdir -p /tmp/mongodb-upgrade && cd /tmp/mongodb-upgrade

curl -O ${BASE}/mongodb-org-server-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-org-mongos-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-org-tools-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-org-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
# Also download mongodb-database-tools and mongodb-mongosh if updated

# Step 2: Transfer RPMs to the target server
scp /tmp/mongodb-upgrade/*.rpm admin@target-vm:/tmp/mongodb-upgrade/

# Step 3: On the target server — stop, upgrade, start
sudo systemctl stop mongod

# Method A: Using rpm -Uvh (upgrade mode)
sudo rpm -Uvh /tmp/mongodb-upgrade/*.rpm

# Method B: Using dnf localinstall (better dependency handling)
sudo dnf localinstall -y /tmp/mongodb-upgrade/*.rpm

sudo systemctl start mongod

# Step 4: Verify
mongod --version
rpm -qa | grep mongo | sort
```

TIP

The 'rpm -Uvh' flag means Upgrade-Verbose-Hash. It replaces older RPMs with newer ones.
Use 'rpm -ivh' only for first-time installs; use 'rpm -Uvh' for upgrades.

11.4 Major Version Upgrade (e.g., 6.0 → 7.0)

Major upgrades require careful planning because they may involve schema changes, driver compatibility issues, and feature compatibility version (FCV) transitions.

11.4.1 Major Upgrade via DNF Repository Switch

```
# Step 1: Verify current FCV matches your running major version
mongosh --eval 'db.adminCommand({getParameter:1, featureCompatibilityVersion:1})'
# Output should show: { featureCompatibilityVersion: { version: "6.0" } }

# Step 2: Ensure FCV is set to current version (not a prior one)
mongosh --eval 'db.adminCommand({setFeatureCompatibilityVersion: "6.0"})'

# Step 3: Full backup with oplog
mongodump --oplog --gzip \
    --out=/apps/mongodb/backups/pre_major_upgrade_$(date +%Y%m%d)

# Step 4: Remove the old repo and create the new one
sudo rm /etc/yum.repos.d/mongodb-org-6.0.repo

sudo tee /etc/yum.repos.d/mongodb-org-7.0.repo <<'EOF'
[mongodb-org-7.0]
name=MongoDB Repository
baseurl=https://repo.mongodb.org/yum/redhat/$releasever/mongodb-org/7.0/x86_64/
gpgcheck=1
enabled=1
gpgkey=https://pgp.mongodb.com/server-7.0.asc
EOF

# Step 5: If version pinning was set, temporarily remove it
sudo dnf versionlock delete mongodb-org*
# Or remove the exclude line from /etc/yum.conf

# Step 6: Stop and upgrade
sudo systemctl stop mongod
sudo dnf clean all
sudo dnf install -y mongodb-org
sudo systemctl start mongod

# Step 7: Verify new version is running
mongod --version

# Step 8: After ALL replica set members are upgraded, set new FCV
mongosh --eval 'db.adminCommand({setFeatureCompatibilityVersion: "7.0"})'

# Step 9: Re-enable version pinning
```

```
sudo dnf versionlock add mongodb-org*
```

11.4.2 Major Upgrade via Manual RPM (Air-Gapped)

```
# Step 1-3: Same as above (check FCV, set FCV, backup)

# Step 4: Download new major version RPMs externally
MONGO_VER="7.0.12"
RHEL_VER="9"
BASE="https://repo.mongodb.org/yum/redhat/${RHEL_VER}/mongodb-org/7.0/x86_64/RPMS"
cd /tmp/mongodb-major-upgrade

curl -O ${BASE}/mongodb-org-server-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-org-mongos-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-org-tools-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-org-${MONGO_VER}-1.el${RHEL_VER}.x86_64.rpm
curl -O ${BASE}/mongodb-database-tools-<ver>.x86_64.rpm
curl -O ${BASE}/mongodb-mongosh-<ver>.x86_64.rpm

# Step 5: Transfer to target, stop, and upgrade
scp *.rpm admin@target-vm:/tmp/mongodb-major-upgrade/

# On target server:
sudo systemctl stop mongod

# Remove old packages first (config files are preserved)
sudo rpm -e --nodeps mongodb-org mongodb-org-server mongodb-org-mongos \
    mongodb-org-tools mongodb-database-tools mongodb-mongosh

# Install new major version
sudo dnf localinstall -y /tmp/mongodb-major-upgrade/*.rpm
sudo systemctl start mongod

# Step 6: Verify and set new FCV
mongod --version
mongosh --eval 'db.adminCommand({setFeatureCompatibilityVersion: "7.0"})'
```

WARNING

Never skip major versions. Upgrade sequentially: 5.0 -> 6.0 -> 7.0.

Always check the official MongoDB upgrade documentation for breaking changes.

Ensure ALL replica set members are on the new version before setting the new FCV.

11.5 Rollback an RPM Upgrade

```
# If an upgrade causes issues, rollback using dnf history
# List recent dnf transactions
sudo dnf history list
```

```
# Undo the last transaction (the upgrade)
sudo dnf history undo last

# Or for manual RPM installs, keep old RPMs and downgrade:
sudo systemctl stop mongod
sudo rpm -Uvh --oldpackage /tmp/mongodb-old-rpms/*.rpm
sudo systemctl start mongod

# IMPORTANT: If you set a new FCV, you MUST downgrade FCV before
# downgrading binaries (when the new version is still running):
mongosh --eval 'db.adminCommand({setFeatureCompatibilityVersion: "6.0"})'
```

11.6 RPM Package Management Reference

Task	Command
List installed MongoDB RPMs	rpm -qa | grep mongo | sort
Check version of mongod RPM	rpm -qi mongodb-org-server
List files in a package	rpm -ql mongodb-org-server
Verify package integrity	rpm -V mongodb-org-server
Check which package owns a file	rpm -qf /usr/bin/mongod
Check available updates (repo)	dnf check-update mongodb-org*
Upgrade all MongoDB RPMs (repo)	dnf update -y mongodb-org
Upgrade via local RPMs	dnf localinstall -y /path/*.rpm
Upgrade via rpm directly	rpm -Uvh /path/*.rpm
First install via rpm	rpm -ivh /path/*.rpm
Remove a package (keep config)	rpm -e mongodb-org-server
Force remove ignoring deps	rpm -e --nodeps mongodb-org-server
Pin versions (prevent auto-upgrade)	dnf versionlock add mongodb-org*
Unpin versions	dnf versionlock delete mongodb-org*
List pinned packages	dnf versionlock list
Undo last dnf transaction	dnf history undo last

12 Quick Reference — Essential Commands Cheat Sheet

12.1 CRUD Operations Quick Reference

```
# INSERT
db.coll.insertOne({ key: "value" })
db.coll.insertMany([{ a: 1 }, { a: 2 }])

# FIND (SELECT)
db.coll.find()                                    // All documents
db.coll.find({ status: "active" })                // Filter
db.coll.find({ age: { $gte: 18 } })              // Comparison
db.coll.find({}, { name: 1, email: 1, _id: 0 })  // Projection
db.coll.find().sort({ created: -1 }).limit(10)    // Sort + Limit
db.coll.countDocuments({ status: "active" })       // Count
db.coll.distinct("status")                         // Unique values

# UPDATE
db.coll.updateOne(
    { _id: ObjectId("...") },
    { $set: { status: "inactive" }, $inc: { counter: 1 } }
)
db.coll.updateMany(
    { status: "pending" },
    { $set: { status: "cancelled" } }
)
db.coll.replaceOne(
    { _id: ObjectId("...") },
    { name: "New", status: "active" }    // Replaces entire document
)

# DELETE
db.coll.deleteOne({ _id: ObjectId("...") })
db.coll.deleteMany({ status: "cancelled" })

# UPSERT (insert if not exists)
db.coll.updateOne(
    { email: "user@example.com" },
    { $set: { name: "User", last_login: new Date() } },
    { upsert: true }
)
```

12.2 Aggregation Pipeline Quick Reference

```
db.orders.aggregate([
    { $match: { status: "shipped" } },                    // Filter
    { $lookup: {                                           // JOIN
        from: "customers", localField: "customer_id",
        foreignField: "_id", as: "customer"
```

```
    }},
    { $unwind: "$customer" },                              // Flatten array
    { $group: {                                            // GROUP BY
        _id: "$customer.region",
        total: { $sum: "$total" },
        count: { $sum: 1 },
        avg_order: { $avg: "$total" }
    }},
    { $sort: { total: -1 } },                             // ORDER BY
    { $limit: 10 },                                       // LIMIT
    { $project: {                                          // SELECT fields
        region: "$_id", total: 1, count: 1,
        avg_order: { $round: ["$avg_order", 2] }
    }}
])
```

12.3 Administrative Commands Quick Reference

Task	Command
Show databases	show dbs
Switch database	use dbname
Show collections	show collections
Server status	db.serverStatus()
Current operations	db.currentOp({ active: true })
Kill an operation	db.killOp(opid)
Replica set status	rs.status()
Profiler status	db.getProfilingStatus()
Enable profiler (slow)	db.setProfilingLevel(1, { slowms: 100 })
Collection stats	db.collection.stats()
Compact collection	db.collection.compact()
Repair database	mongod --repair --dbpath /apps/mongodb/data
Export collection	mongoexport --collection=X --out=X.json
Import collection	mongoimport --collection=X --file=X.json
Full dump	mongodump --out=/apps/mongodb/backups/
Full restore	mongorestore /apps/mongodb/backups/
Create user	db.createUser({...})
Show users	db.getUsers()

Rotate logs	db.adminCommand({ logRotate: 1 })

12.4 Comparison Operators Quick Reference

Operator	Description	Example
$eq	Equals	{ status: { $eq: "active" } }
$ne	Not equal	{ status: { $ne: "deleted" } }
$gt / $gte	Greater than / or equal	{ age: { $gte: 18 } }
$lt / $lte	Less than / or equal	{ price: { $lt: 100 } }
$in	In array	{ status: { $in: ["a","b"] } }
$nin	Not in array	{ status: { $nin: ["deleted"] } }
$regex	Pattern match	{ name: { $regex: /^A/i } }
$exists	Field exists	{ phone: { $exists: true } }
$and	Logical AND	{ $and: [{...},{...}] }
$or	Logical OR	{ $or: [{...},{...}] }
$not	Negation	{ age: { $not: { $lt: 18 } } }

12.5 Important File Paths (/apps Layout)

Path	Purpose
/etc/mongod.conf	Main configuration file
/apps/mongodb/data	Database data files (dbPath)
/apps/mongodb/log/mongod.log	Server log file
/apps/mongodb/backups	Backup staging directory
/apps/mongodb/keyfile/mongo-keyfile	Replica set auth keyfile
/apps/mongodb/ssl/	TLS certificate and key files
/apps/mongodb/scripts/	Admin scripts (backup, health check)
/etc/yum.repos.d/mongodb-org-7.0.repo	MongoDB YUM repository config
/etc/systemd/system/mongod.service.d/	Systemd overrides
/etc/logrotate.d/mongodb	Log rotation config

13 User and Role Management — Day-to-Day Operations

13.1 Listing Users and Roles

```
# List all users in the current database
db.getUsers()

# List all users across ALL databases (run from admin)
use admin
db.system.users.find().pretty()

# Get a specific user's details
db.getUser("app_user")

# Show user details including privileges
db.getUser("app_user", { showPrivileges: true })

# List all roles in the current database
db.getRoles()

# List all roles including built-in roles
db.getRoles({ showBuiltinRoles: true })

# Get a specific role's details with privileges
db.getRole("orderManager", { showPrivileges: true })
```

13.2 Creating Users

```
# Create a read-write user for a specific database
use appdb
db.createUser({
    user: "app_user",
    pwd: passwordPrompt(),     // Secure interactive prompt
    roles: [
        { role: "readWrite", db: "appdb" }
    ],
    mechanisms: ["SCRAM-SHA-256"]  // Preferred auth mechanism
})

# Create a user with multiple database access
use admin
db.createUser({
    user: "multi_db_user",
    pwd: "securePass!",
    roles: [
        { role: "readWrite", db: "appdb" },
        { role: "read", db: "logsdb" },
        { role: "readWrite", db: "staging_db" }
    ]
```

```
})

# Create a DBA admin user
use admin
db.createUser({
    user: "dba_admin",
    pwd: passwordPrompt(),
    roles: [
        { role: "userAdminAnyDatabase", db: "admin" },
        { role: "readWriteAnyDatabase", db: "admin" },
        { role: "dbAdminAnyDatabase", db: "admin" },
        { role: "clusterAdmin", db: "admin" }
    ]
})
```

13.3 Modifying Users — Password Changes, Role Updates

```
# Change a user's password
use appdb
db.changeUserPassword("app_user", passwordPrompt())

# Or with a direct string (less secure — visible in shell history)
db.changeUserPassword("app_user", "newSecurePass123!")

# Grant additional roles to an existing user
db.grantRolesToUser("app_user", [
    { role: "dbAdmin", db: "appdb" },
    { role: "read", db: "logsdb" }
])

# Revoke roles from a user
db.revokeRolesFromUser("app_user", [
    { role: "read", db: "logsdb" }
])

# Update a user completely (replaces all roles)
db.updateUser("app_user", {
    roles: [
        { role: "readWrite", db: "appdb" },
        { role: "readWrite", db: "staging_db" }
    ],
    mechanisms: ["SCRAM-SHA-256"]
})
```

13.4 Dropping Users

```
# Drop a specific user
use appdb
db.dropUser("app_user")
```

```
# Drop all users from a database
db.dropAllUsers()

# Verify user has been removed
db.getUsers()
```

> **WARNING**
> Dropping the last admin user with userAdminAnyDatabase will lock you out.
> Always ensure at least one admin user exists in the admin database before dropping others.

13.5 Custom Roles — Create, Modify, Drop

```
# Create a custom role
use appdb
db.createRole({
    role: "reportAnalyst",
    privileges: [
        {
            resource: { db: "appdb", collection: "" },  // All collections
            actions: ["find", "listCollections", "collStats", "dbStats"]
        },
        {
            resource: { db: "appdb", collection: "system.profile" },
            actions: ["find"]    // Allow reading profiler data
        }
    ],
    roles: []   // No inherited roles
})

# Grant additional privileges to an existing custom role
db.grantPrivilegesToRole("reportAnalyst", [
    {
        resource: { db: "logsdb", collection: "" },
        actions: ["find"]
    }
])

# Revoke privileges from a role
db.revokePrivilegesFromRole("reportAnalyst", [
    {
        resource: { db: "logsdb", collection: "" },
        actions: ["find"]
    }
])

# Drop a custom role
db.dropRole("reportAnalyst")
```

13.6 Authentication Troubleshooting

```
# Test authentication from the command line
mongosh -u app_user -p --authenticationDatabase appdb

# Check if auth is enabled
db.adminCommand({ getParameter: 1, authenticationMechanisms: 1 })

# View failed authentication attempts in the log
grep "Authentication failed" /apps/mongodb/log/mongod.log | tail -10

# If locked out — restart without auth to reset users:
# 1. Stop mongod
sudo systemctl stop mongod
# 2. Comment out security.authorization in mongod.conf
# 3. Start mongod without auth
sudo systemctl start mongod
# 4. Connect and recreate admin user
mongosh
use admin
db.createUser({ user:"admin", pwd:"newpass", roles:["root"] })
# 5. Re-enable authorization and restart
sudo systemctl restart mongod
```

14 Storage and Space Management

14.1 Understanding MongoDB Storage

MongoDB's WiredTiger storage engine stores data in compressed format. Deleted documents do not immediately release disk space — the freed space is reused internally. Understanding storage metrics is critical for capacity planning.

```
# Database-level storage stats
use appdb
db.stats()
// Key fields:
//   dataSize    — actual size of data (uncompressed)
//   storageSize — disk space allocated for data (after compression)
//   indexSize   — total size of all indexes
//   totalSize   — storageSize + indexSize
//   fsTotalSize — total filesystem size
//   fsUsedSize  — used filesystem space

# Collection-level storage stats
db.orders.stats()
// Key fields:
//   size            — uncompressed data size
//   storageSize     — actual disk usage (compressed)
//   totalIndexSize  — disk used by all indexes on this collection
//   count           — document count
//   avgObjSize      — average document size in bytes

# Human-readable sizes
db.orders.stats({ scale: 1048576 })  // Sizes in MB

# Quick collection size summary for all collections
db.getCollectionNames().forEach(function(c) {
    var s = db[c].stats()
    print(c + ": " +
        (s.storageSize/1048576).toFixed(2) + " MB data, " +
        (s.totalIndexSize/1048576).toFixed(2) + " MB indexes, " +
        s.count + " docs")
})
```

14.2 Compaction — Reclaiming Disk Space

After heavy delete or update operations, collections may have fragmented storage. The compact command defragments and reclaims unused space within data files.

```
# Compact a collection (blocks reads/writes on that collection)
db.orders.compact()

# Compact with force (useful if errors occur)
db.runCommand({ compact: "orders", force: true })
```

```
# Compact all collections in a database
db.getCollectionNames().forEach(function(c) {
    if (!c.startsWith("system.")) {
        print("Compacting: " + c)
        db.runCommand({ compact: c })
    }
})

# Monitor disk usage before and after compaction
db.orders.stats({ scale: 1048576 })
```

WARNING

compact blocks all operations on the collection during execution.
Run during maintenance windows. On replica sets, run on secondaries first,
then step down and compact the former primary.

14.3 Disk Space Monitoring

```
# OS-level disk usage
df -h /apps/mongodb/data

# Detailed du on MongoDB data directory
du -sh /apps/mongodb/data/*

# MongoDB-level: database sizes across the server
db.adminCommand("listDatabases")

# Script: compare allocated vs. actual storage
db.adminCommand("listDatabases").databases.forEach(function(d) {
    var s = db.getSiblingDB(d.name).stats()
    var waste = s.storageSize - s.dataSize
    print(d.name + ": " +
        "data=" + (s.dataSize/1048576).toFixed(1) + "MB " +
        "storage=" + (s.storageSize/1048576).toFixed(1) + "MB " +
        "waste=" + (waste/1048576).toFixed(1) + "MB")
})
```

14.4 Dropping and Cleaning Up Unused Data

```
# Drop an entire collection (immediate space reclamation)
db.old_logs.drop()

# Drop a database
use obsolete_db
db.dropDatabase()

# Delete old documents (space reused internally, not freed to OS)
```

```
db.app_logs.deleteMany({
    timestamp: { $lt: ISODate("2024-01-01T00:00:00Z") }
})

# To actually reclaim space after mass deletes, run compact
db.app_logs.compact()

# Use TTL indexes for automatic expiration (preferred for logs)
db.app_logs.createIndex(
    { timestamp: 1 },
    { expireAfterSeconds: 7776000 }    // Auto-delete after 90 days
)
```

14.5 WiredTiger Storage Internals

```
# WiredTiger cache statistics
db.serverStatus().wiredTiger.cache

# Key metrics to monitor:
//   "bytes currently in the cache"
//   "maximum bytes configured"
//   "tracked dirty bytes in the cache"   — pending writes
//   "pages evicted"                       — cache pressure indicator

# WiredTiger block manager stats (disk I/O)
db.serverStatus().wiredTiger["block-manager"]

# Check compression ratio for a collection
var stats = db.orders.stats()
var ratio = stats.size / stats.storageSize
print("Compression ratio: " + ratio.toFixed(2) + ":1")
print("Uncompressed: " + (stats.size/1048576).toFixed(2) + " MB")
print("On disk: " + (stats.storageSize/1048576).toFixed(2) + " MB")

# Change compression for a new collection
db.createCollection("events", {
    storageEngine: {
        wiredTiger: {
            configString: "block_compressor=zstd"  // zstd, snappy, zlib, none
        }
    }
})
```

14.6 Repairing a Corrupted Database

```
# Method 1: mongod --repair (OFFLINE — must stop mongod first)
sudo systemctl stop mongod
mongod --repair --dbpath /apps/mongodb/data
sudo chown -R mongod:mongod /apps/mongodb/data
sudo systemctl start mongod
```

```
# Method 2: validate command (ONLINE — check for corruption)
db.orders.validate({ full: true })
// Look for: valid: true, errors: []

# Validate all collections in a database
db.getCollectionNames().forEach(function(c) {
    var result = db[c].validate({ full: true })
    if (!result.valid) {
        print("CORRUPTION DETECTED: " + c)
        printjson(result.errors)
    } else {
        print("OK: " + c)
    }
})
```

WARNING

mongod --repair is a last resort. It rebuilds all data files and indexes.
Always take a filesystem backup of /apps/mongodb/data BEFORE running repair.
Repair may take a long time on large databases.

15 Transactions — Multi-Document ACID Operations

15.1 Transaction Overview

Starting with MongoDB 4.0, multi-document ACID transactions are supported on replica sets. MongoDB 4.2 extended support to sharded clusters. Transactions guarantee atomicity across multiple documents and collections — either all operations commit or all roll back.

Feature	Requirement	Notes
Single-document atomicity	Always available	Every single-document write is atomic by default
Multi-document transactions	Replica set required	Not available on standalone (use replSet even with 1 node)
Cross-collection transactions	MongoDB 4.0+	Within the same database
Cross-database transactions	MongoDB 4.2+	Across different databases
Sharded transactions	MongoDB 4.2+	Across sharded collections
Max transaction duration	Default 60 seconds	Configurable via transactionLifetimeLimitSeconds

15.2 Using Transactions in mongosh

```
// Start a session and transaction
var session = db.getMongo().startSession()
session.startTransaction({
    readConcern: { level: "snapshot" },
    writeConcern: { w: "majority" },
    readPreference: "primary"
})

var ordersDB = session.getDatabase("appdb")

try {
    // Perform multiple operations atomically
    ordersDB.orders.insertOne({
        order_id: "ORD-5001",
        customer_id: ObjectId("64a..."),
        total: 299.99,
        status: "confirmed"
    })

    ordersDB.inventory.updateOne(
        { sku: "WIDGET-A", qty: { $gte: 1 } },
        { $inc: { qty: -1 }, $push: { reservations: "ORD-5001" }}
    )

    ordersDB.accounts.updateOne(
        { customer_id: ObjectId("64a...") },
```

```
        { $inc: { balance: -299.99 }}
    )

    // All succeeded — commit
    session.commitTransaction()
    print("Transaction committed successfully")

} catch (error) {
    // Any failure — abort all changes
    session.abortTransaction()
    print("Transaction aborted: " + error.message)

} finally {
    session.endSession()
}
```

15.3 Transaction Configuration and Tuning

```
# View current transaction timeout
db.adminCommand({ getParameter: 1, transactionLifetimeLimitSeconds: 1 })

# Increase timeout (default 60 seconds)
db.adminCommand({ setParameter: 1, transactionLifetimeLimitSeconds: 120 })

# View active transactions
db.currentOp({ "transaction": { $exists: true } })

# Kill a stuck transaction
db.currentOp({ "transaction": { $exists: true } }).inprog.forEach(function(op) {
    if (op.secs_running > 30) {
        print("Killing transaction opid:", op.opid)
        db.killOp(op.opid)
    }
})

# Monitor transaction metrics
db.serverStatus().transactions
//   totalStarted, totalCommitted, totalAborted
//   currentActive, currentOpen
```

TIP

Design your schema to minimize the need for multi-document transactions. MongoDB's document model allows embedding related data in a single document, which is atomic by default and much faster than transactions.

16 Common Errors and Troubleshooting Runbook

16.1 Startup and Service Failures

Error: mongod fails to start — Address already in use

```
# Symptom: mongod.service failed; "Address already in use"
# Cause: Another mongod instance or process using port 27017

# Check what's using the port
sudo ss -tlnp | grep 27017
sudo lsof -i :27017

# Kill the rogue process or change port in mongod.conf
# If stale PID/socket file:
sudo rm -f /apps/mongodb/data/mongod.lock
sudo rm -f /tmp/mongodb-27017.sock
sudo systemctl start mongod
```

Error: mongod fails to start — Permission denied on data directory

```
# Symptom: "Failed to open file /apps/mongodb/data/..." permission denied
# Cause: Incorrect ownership after manual file operations

# Fix ownership
sudo chown -R mongod:mongod /apps/mongodb/data
sudo chown -R mongod:mongod /apps/mongodb/log
sudo chmod 750 /apps/mongodb/data

# Fix SELinux labels
sudo restorecon -Rv /apps/mongodb/data
sudo restorecon -Rv /apps/mongodb/log
sudo systemctl start mongod
```

Error: mongod fails to start — WiredTiger cannot open data files

```
# Symptom: "WiredTiger error ... cannot open" or "data files corruption"
# Cause: Unclean shutdown, disk errors, or storage corruption

# Step 1: Try normal start first
sudo systemctl start mongod

# Step 2: If still failing, check the log
tail -50 /apps/mongodb/log/mongod.log

# Step 3: Run repair (TAKE BACKUP OF DATA DIR FIRST!)
sudo systemctl stop mongod
sudo cp -a /apps/mongodb/data /apps/mongodb/data_backup_$(date +%Y%m%d)
```

```
mongod --repair --dbpath /apps/mongodb/data
sudo chown -R mongod:mongod /apps/mongodb/data
sudo systemctl start mongod
```

Error: mongod fails to start on boot — systemd timeout

```
# Symptom: mongod.service timed out during boot on large databases
# Cause: Default systemd TimeoutStartSec too short for large journal recovery

# Fix: Create systemd override
sudo mkdir -p /etc/systemd/system/mongod.service.d
sudo tee /etc/systemd/system/mongod.service.d/timeout.conf <<'EOF'
[Service]
TimeoutStartSec=300
TimeoutStopSec=300
EOF

sudo systemctl daemon-reload
sudo systemctl start mongod
```

16.2 Connection and Authentication Errors

Error: MongoServerError — Authentication failed

```
# Symptom: "Authentication failed" when connecting
# Possible causes:

# 1. Wrong authenticationDatabase (most common!)
# WRONG (defaults to "test"):
mongosh -u admin -p mypass
# CORRECT:
mongosh -u admin -p mypass --authenticationDatabase admin

# 2. User does not exist in the specified database
mongosh --eval 'db.getSiblingDB("admin").getUsers()'

# 3. Wrong password — reset it:
mongosh -u admin -p --authenticationDatabase admin
use appdb
db.changeUserPassword("app_user", "newPassword!")

# 4. SCRAM mechanism mismatch
# Check user's mechanisms:
use admin
db.getUser("admin", { showCredentials: true })
```

Error: MongoNetworkError — connection refused

```
# Symptom: "connect ECONNREFUSED 127.0.0.1:27017"
# Step 1: Check if mongod is running
sudo systemctl status mongod
pgrep mongod

# Step 2: Check if mongod is listening on the expected port
sudo ss -tlnp | grep 27017

# Step 3: Check bindIp in mongod.conf
grep bindIp /etc/mongod.conf
# Ensure 0.0.0.0 or the correct IP is listed for remote access

# Step 4: Check firewall
sudo firewall-cmd --list-ports | grep 27017

# Step 5: Check mongod log for errors
tail -30 /apps/mongodb/log/mongod.log
```

16.3 Storage and Disk Errors

Error: No space left on device

```
# Symptom: Write failures, "No space left on device"

# Immediate actions:
# 1. Check disk usage
df -h /apps/mongodb/data

# 2. Identify large collections
mongosh --eval '
    db.getSiblingDB("appdb").getCollectionNames().forEach(function(c) {
        var s = db.getSiblingDB("appdb")[c].stats({scale:1048576})
        print(c + ": " + s.storageSize.toFixed(0) + " MB")
    })
'

# 3. Quick space recovery options:
# a. Drop temp/test databases
mongosh --eval 'db.getSiblingDB("test").dropDatabase()'

# b. Remove old backups
ls -lh /apps/mongodb/backups/
rm -rf /apps/mongodb/backups/old_backup_*

# c. Rotate and compress logs
db.adminCommand({ logRotate: 1 })
gzip /apps/mongodb/log/mongod.log.2025*

# d. Compact collections (reclaims internal fragmentation)
mongosh --eval 'db.getSiblingDB("appdb").large_collection.compact()'

# 4. Long-term: add disk capacity or move data to larger volume
```

16.4 Replication Errors

Error: Replication lag keeps increasing

```
# Step 1: Measure the lag
mongosh --eval 'rs.printSecondaryReplicationInfo()'

# Step 2: Check oplog window
mongosh --eval 'rs.printReplicationInfo()'
# If oplog window < replication lag, secondary cannot catch up!

# Step 3: Check for long-running operations on secondary
mongosh --host secondary-host --eval '
    db.currentOp({ active: true, secs_running: { $gt: 10 }})
'

# Step 4: Check network bandwidth between nodes
ping primary-host
iperf3 -c primary-host

# Step 5: Remediation options:
# a. Increase oplog size
db.adminCommand({ replSetResizeOplog: 1, size: 4096 })  // 4 GB

# b. If secondary is too far behind, resync:
# On the secondary — stop, remove data, restart
sudo systemctl stop mongod
sudo rm -rf /apps/mongodb/data/*
sudo systemctl start mongod
# Secondary will perform initial sync automatically
```

Error: Replica set member in RECOVERING state

```
# Symptom: rs.status() shows a member as RECOVERING

# Cause 1: Secondary fell behind oplog window
# Fix: Force initial sync
# On the RECOVERING member:
sudo systemctl stop mongod
sudo rm -rf /apps/mongodb/data/*
sudo systemctl start mongod

# Cause 2: Disk I/O too slow to apply oplog entries
# Fix: Check I/O wait, consider faster storage
iostat -xz 2 5

# Cause 3: Index build in progress
# Fix: Wait for it to complete; check progress:
db.currentOp({ "msg": /Index Build/ })
```

16.5 Performance-Related Errors

Error: Slow query warnings flooding the log

```
# Symptom: Thousands of "Slow query" entries in mongod.log
# The default slowOpThresholdMs is 100ms

# Step 1: Identify the worst offenders
db.setProfilingLevel(1, { slowms: 200 })
db.system.profile.find().sort({ millis: -1 }).limit(5).pretty()

# Step 2: Check for missing indexes (COLLSCAN)
db.system.profile.find({ planSummary: "COLLSCAN" }).sort({ ts: -1 })

# Step 3: Create indexes for common query patterns
db.orders.createIndex({ customer_id: 1, order_date: -1 })

# Step 4: Adjust threshold if too noisy (raise to 500ms)
db.setProfilingLevel(1, { slowms: 500 })
# Or in mongod.conf:
# operationProfiling:
#   mode: slowOp
#   slowOpThresholdMs: 500
```

Error: Out of memory / OOM killer invoked

```
# Symptom: mongod killed by Linux OOM killer
# Evidence: "Out of memory: Kill process" in /var/log/messages

# Step 1: Check current memory usage
free -h
cat /proc/$(pgrep mongod)/status | grep -i vmrss

# Step 2: Check WiredTiger cache setting
mongosh --eval 'db.serverStatus().wiredTiger.cache["maximum bytes configured"]'

# Step 3: Reduce WiredTiger cache size
# In mongod.conf:
# storage.wiredTiger.engineConfig.cacheSizeGB: <50% of RAM minus 1 GB>
# Example for 8 GB RAM server: cacheSizeGB: 3

# Step 4: Dynamic adjustment (no restart)
db.adminCommand({ setParameter: 1, wiredTigerCacheSizeGB: 3 })

# Step 5: Ensure vm.swappiness is set low
sysctl vm.swappiness
# Should be 1; if not:
echo "vm.swappiness = 1" | sudo tee -a /etc/sysctl.d/99-mongodb.conf
sudo sysctl -p /etc/sysctl.d/99-mongodb.conf
```

16.6 Common Errors Quick Reference

Error / Symptom	Likely Cause	Quick Fix

Address already in use	Stale PID/socket or duplicate process	rm mongod.lock + /tmp socket; kill rogue process
Permission denied on data	Ownership changed (manual cp, restore)	chown -R mongod:mongod /apps/mongodb/data
Authentication failed	Wrong --authenticationDatabase	Add --authenticationDatabase admin
Connection refused	mongod not running or wrong bindIp	systemctl start mongod; check bindIp
No space left on device	Data growth or backup accumulation	Drop temp DBs; compact; rotate logs; expand disk
Replication lag increasing	Oplog too small or slow secondary I/O	Increase oplog; check network; resync secondary
RECOVERING state	Secondary fell behind oplog window	Stop, wipe data, restart for fresh initial sync
COLLSCAN slow queries	Missing indexes	Create compound index per ESR rule
OOM kill	WiredTiger cache too large for RAM	Reduce cacheSizeGB to 50% RAM minus 1 GB
WiredTiger data corruption	Unclean shutdown or disk failure	Backup data dir; run mongod --repair
Cursor not found	Server-side cursor timeout (10 min)	Use noCursorTimeout or batch processing
Too many open files	ulimit too low	Set nofile=64000 in limits.d and systemd override

17 Disaster Recovery Scenarios and Runbooks

17.1 Scenario: Accidental Collection Drop

```
# Someone ran: db.critical_data.drop()

# Option A: Restore from latest mongodump backup
mongorestore --db=appdb --collection=critical_data --drop \
    /apps/mongodb/backups/full_latest/appdb/critical_data.bson

# Option B: Point-in-time restore (if oplog backup exists)
# Restore full backup then replay oplog up to just BEFORE the drop
mongorestore --oplogReplay \
    --oplogLimit="<timestamp_before_drop>" \
    --drop \
    /apps/mongodb/backups/full_oplog_latest/

# Option C: Restore from a secondary (if replica set, act FAST)
# If secondary hasn't replicated the drop yet:
mongodump --host=secondary-host --db=appdb \
    --collection=critical_data \
    --out=/tmp/emergency_recover/
mongorestore --db=appdb --collection=critical_data \
    /tmp/emergency_recover/appdb/critical_data.bson
```

17.2 Scenario: Accidental Database Drop

```
# Someone ran: use critical_db; db.dropDatabase()

# Restore entire database from backup
mongorestore --db=critical_db --drop \
    /apps/mongodb/backups/full_latest/critical_db/

# If using archive backup
mongorestore --gzip --nsInclude="critical_db.*" \
    --archive=/apps/mongodb/backups/full_latest.archive
```

17.3 Scenario: Accidental Mass Delete / Update

```
# Someone ran: db.orders.deleteMany({}) or wrong update filter

# Step 1: Assess the damage
db.orders.countDocuments({})
db.orders.find().limit(5).pretty()

# Step 2: Restore from backup to a temporary database
```

```
mongorestore --db=recovery_db \
    --collection=orders \
    /apps/mongodb/backups/full_latest/appdb/orders.bson

# Step 3: Verify recovery data
mongosh --eval 'db.getSiblingDB("recovery_db").orders.countDocuments({})'

# Step 4: Copy recovered data back to production
mongosh --eval '
    db.getSiblingDB("recovery_db").orders.aggregate([
        { $merge: {
            into: { db: "appdb", coll: "orders" },
            whenMatched: "keepExisting",
            whenNotMatched: "insert"
        }}
    ])
'

# Step 5: Cleanup
mongosh --eval 'db.getSiblingDB("recovery_db").dropDatabase()'
```

17.4 Scenario: Primary Server Total Loss (Replica Set)

```
# Primary VM destroyed or unrecoverable

# Step 1: Check replica set status from a surviving member
mongosh --host secondary1 --eval 'rs.status()'
# A secondary should auto-elect as new primary

# Step 2: If no automatic election (no majority)
# Force reconfiguration from a surviving member:
cfg = rs.conf()
cfg.members = cfg.members.filter(m => m.host !== "dead-primary:27017")
rs.reconfig(cfg, { force: true })

# Step 3: Build a replacement node
# Install MongoDB, configure, start with same replSetName
# Add to the replica set:
rs.add("new-node.internal:27017")
# It will automatically perform initial sync

# Step 4: Once synced, rebalance priorities
cfg = rs.conf()
cfg.members.forEach(function(m) {
    if (m.host === "new-node.internal:27017") m.priority = 10
})
rs.reconfig(cfg)
```

17.5 Scenario: Standalone Server Total Loss (No Replica Set)

```
# Standalone mongod — server destroyed

# Step 1: Build replacement VM with same AlmaLinux version
# Step 2: Install MongoDB (same version as original)
# Step 3: Configure mongod.conf identically (same paths under /apps)
# Step 4: Restore from latest backup

# From directory backup:
mongorestore --drop /apps/mongodb/backups/full_latest/

# From archive:
mongorestore --drop --gzip \
    --archive=/apps/mongodb/backups/full_latest.archive

# From filesystem snapshot:
# Mount snapshot and copy data files back:
sudo systemctl stop mongod
sudo cp -a /mnt/snapshot/apps/mongodb/data/* /apps/mongodb/data/
sudo chown -R mongod:mongod /apps/mongodb/data
sudo systemctl start mongod

# Step 5: Validate data integrity
mongosh --eval '
    db.adminCommand("listDatabases").databases.forEach(function(d) {
        var mydb = db.getSiblingDB(d.name)
        mydb.getCollectionNames().forEach(function(c) {
            var r = mydb[c].validate({full:true})
            print(d.name + "." + c + ": " + (r.valid ? "OK" : "CORRUPT"))
        })
    })
'
```

17.6 Disaster Recovery Checklist

Priority	Action	Details
P0	Verify backups exist and are readable	mongorestore --dryRun /path/backup/ (or inspect .bson files)
P0	Test restore to a non-production environment	Schedule quarterly restore drills
P1	Document RPO and RTO targets	RPO = max data loss tolerable; RTO = time to recover
P1	Maintain offsite backup copies	Copy backups to separate storage/region daily
P1	Use replica sets (min 3 members)	Automatic failover = near-zero RTO for node failures
P2	Enable oplog for PITR capability	Allows recovery to any point within oplog window
P2	Monitor backup job success daily	Alert on backup script failures via cron + email
P2	Keep N-1 MongoDB RPMs available	Enables quick rollback if upgrade fails
P3	Document all connection strings	Application configs, driver URIs, admin credentials

P3	Maintain server build runbook	Automate VM provisioning for fast replacement builds